The PhpServicePlatform Kernel

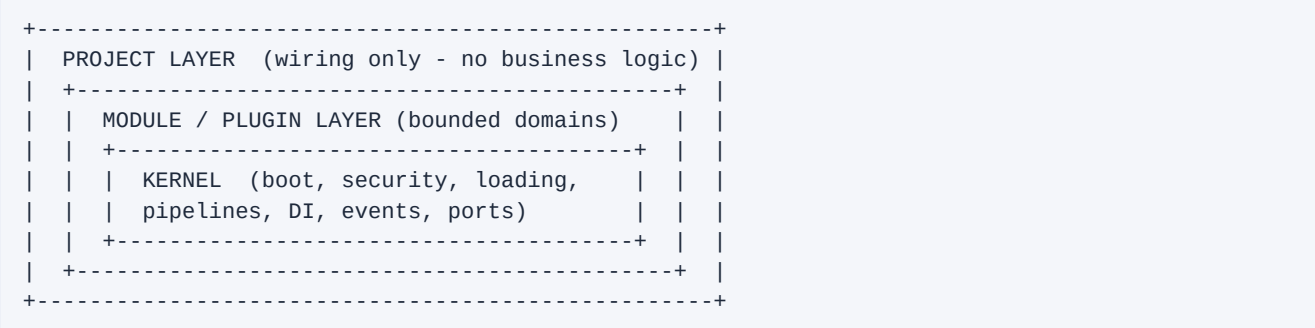
A complete, file-by-file walkthrough of the Gated Demand Architecture (GDA) core - flow, interconnections, and every public method.

Package: alfacode-team/php-service-platform
Namespace: AlfacodeTeam\PhpServicePlatform\Kernel\
Source root: src/Kernel/
PHP: 8.2+ | Generated: 2026-06-24

How to read this document

The kernel knows nothing about any business domain. It boots, secures, loads only the modules a given request needs, runs a pipeline, and gets out of the way. This guide follows the actual runtime order: configuration -> build -> materialize -> a live request. Each subsystem section lists its files, what they do, how they connect to neighbours, and the full purpose of every public method.

The three worlds (where the kernel sits)



- Kernel - changes rarely; defines port interfaces and the request machinery.
- Module/Plugin - one bounded domain each; wires to the kernel through contracts only.
- Project - knows everything, contains no business logic; supplies port implementations.

1. The End-to-End Lifecycle

Everything the kernel does happens in three phases. Understanding these phases is the key to understanding every file that follows.

Phase A - configure() and build() (compile-only)

Kernel::configure() returns a fluent builder. You declare ports, security layers, modules, the error pipeline and project routes, then call build(). build() runs the BootPipeline: it validates config and compiles the manifests to disk. It does NOT construct pipelines or wire modules - a process that only serves HTTP never pays to build the worker surface.

```
$kernel = Kernel::configure()
    ->withPorts([DatabasePort::class => new MySQLAdapter(...)])
    ->withSecurity([new FirewallLayer(...), new CsrfTokenLayer(...)])
    ->withModules([InvoiceModule\Provider::class])
    ->withRoutes(EntryHelpers::projectRoutes($projectPath))
    ->build();   // BootPipeline runs here
```

Phase B - materialize() (first entry-point call)

The first call to http(), cli(), workerLoop() or container() triggers materialize() exactly once. It constructs the three pipelines (cheap shells), wires every module once by calling Provider::boot() on each, exposes kernel services into the CoreContainer, then FREEZES the core container. After freezing, any bind/singleton/instance/extend throws - this is what guarantees Swoole cross-request safety.

Phase C - a live request/job/command

HTTP requests flow through the HttpPipeline onion. The full order built in HttpPipeline::buildStages():

```
ErrorStage            (outermost - catches every Throwable)
  CorrelationIdStage  (X-Correlation-ID)
    SecurityStage     (SecurityGateway -> Identity or deny)
      [after.security hooks]
      ResolveStage    (RouteMatcher -> route_entry + params)
        LoadStage     (dep graph -> OnDemandLoader -> ModuleContainer)
          [after.load hooks]
          RouteFilterStage  (per-route filters: auth, throttle...)
            ExecuteStage     (controller -> Response)
              [after.execute hooks]
```

Each stage receives ($request, $next) and decides whether to call $next. Denials short-circuit the onion, so a request rejected at SecurityStage never reaches LoadStage - zero module cost, the core promise of GDA.

2. Kernel Entry Point

`Kernel.php`

The fluent builder and the single public entry surface. Holds configuration, runs build(), performs lazy materialize(), and hands out the HTTP/CLI/Worker pipelines.

Builder & lifecycle methods

`static configure(): self`

Start a new builder. The only constructor path.

`withBasePath(string $path): self`

Set the application root used by Paths for var/cache/logs resolution.

`withProjectPath(string $path): self`
Pin runtime roots (var/userdata/config) under a specific project directory.

`withPorts(array $bindings): self`
Bind Port interface => adapter. A Closure is a lazy singleton (built on first use); a pre-built object is bound eagerly. Merges with previous calls.

`withSecurity(array $layers): self`
Append SecurityLayerContract instances. Order matters - cheapest first (firewall, rate-limit, CSRF).

`withErrorPipeline(ErrorPipeline|callable $p): self`
Provide the error notifier pipeline; a callable receives the port bindings so notifiers can reuse adapters.

`withModules(array $modules): self`
Append on-demand module providers (deduped, order preserved). Their register() runs only when a route needs them.

`withEssentialModules(array $modules): self`
Append cross-cutting modules registered into EVERY request container (sessions, cookies).

`withRoutes(array $routes): self`
Register project-layer routes compiled under the synthetic '__project__' scope (no module graph).

`build(): self`
Run the BootPipeline (validate + compile manifests). Compile-only; fails fast with BootFailureException.

Entry points (each materializes on first call)

`http(): HttpPipeline`
Materialize for RuntimeMode::Http and return the HTTP pipeline. Call handle(Request).

`cli(): CliPipeline`
Materialize for RuntimeMode::Cli and return the CLI pipeline. Call run(argv).

`workerLoop(): WorkerLoop`
Materialize for RuntimeMode::Worker and return the job loop. Call run(puller).

`container(): CoreContainer`
Materialize (defaults to Cli surface) and return the frozen core container - for tooling/tests.

`mode(): ?RuntimeMode`
The surface this kernel materialized for, or null if no entry point ran yet.

`requestTeardown(): void`
Per-request cleanup hook. No-op today (ModuleContainer is GC'd per request); call it after every Swoole request for forward-compatibility.

3. Boot Subsystem (src/Kernel/Boot)

Invoked once by Kernel::build(). Ten ordered stages, fail-fast. Each stage implements BootStageContract::run(): void and throws BootException on failure; BootPipeline rewraps that as BootFailureException naming the failing stage.

`Boot/BootPipeline.php`

Assembles the ten boot stages in fixed, meaningful order and runs them. Constructor takes the module classes, the CoreContainer, security layers and project routes.

`run(): void`
Execute every stage in order; the first failure aborts the boot with the stage name and reason.

`Boot/Stages/*  (the ten stages, run() each)`

1 ValidateConfig - every env var a module declares is present and typed. 2 DetectConflicts - no two modules share a solves() domain. 3 DetectCycles - no circular requires[] chains. 4 CompileServiceManifest - dependency graph -> service-manifest.php (+ synthetic __project__). 5 CompileRouteManifest - plugin then project routes -> route-manifest.php (project overrides). 6 CompileViewManifest - project-first view cascade + namespaces. 7

CompileJobManifest - jobs with their solving domain. 8 CompileCommandManifest - CLI commands. 9 RegisterPorts - verify Port->Adapter bindings exist in the core. 10 BindSecurity - validate the configured security layers.

CompileRouteManifestStage::PROJECT_SCOPE = '__project__' is the public constant marking project-layer routes that carry no module dependency graph.

Boot/ManifestReader.php / ManifestWriter.php

Read a module's module.json and write compiled manifests to var/cache/manifests/.

`ManifestReader::read(string $moduleClass): array`
Locate and decode the module.json next to a provider class into an array.

`ManifestWriter::write(string $relativePath, array $data): string`
Atomically write a compiled PHP manifest under the cache manifests dir; returns the absolute path.

4. Container Subsystem (DI - bind-it engine)

Both containers extend PHPShots\Common\Container (the bind-it package), gaining reflection autowiring, contextual bindings and PSR-11. The kernel layers GDA scope rules on top.

Container/CoreContainer.php

App-lifetime container, one per worker process. Holds ports and kernel services. Frozen at the end of materialize() - writes after that throw LogicException.

`instance(string $abstract, object $i): void`
Bind an already-built object (e.g. an eager port).

`bind(string $abstract, Closure|string|null $c=null, bool $shared=false): void`
Register a factory binding; honours freeze.

`singleton(string $abstract, Closure|string|null $c=null): void`
Bind a shared lazy factory - built once on first resolution.

`extend($abstract, Closure $closure): void`
Decorate an existing binding before freeze.

`freeze(): void`
Lock the container; all further writes throw. Called during materialize().

`isFrozen(): bool`
True after materialize(), false after build().

`static getInstance(): never / setInstance(...): never`
Disabled - throw LogicException. There is no global singleton (Swoole safety).

Container/ModuleContainer.php

Request-scoped (or per-job) container. New instance per request, discarded after. Enforces the five access rules: internal bindings resolved cross-scope throw ScopeViolationException.

`bind / singleton / instance(...)`
Public bindings - resolvable by any module that declares the domain in requires[].

`bindInternal(string $abstract, Closure $factory): void`
Private binding - throws ScopeViolationException if resolved from outside its owning module scope.

`setScope(string $scope): void`
Set the active solve-domain so subsequent register() bindings are attributed to that module.

`make($abstract, array $parameters=[]): mixed`
Resolve with autowiring, enforcing scope rules.

`makeInScope(string $abstract, string $scope): mixed`
Resolve as if called from a given scope - used by ExecuteStage to instantiate controllers.

`has(string $id): bool / get(string $id): mixed`
PSR-11 accessors.

`reset(): void`
Full lifecycle teardown - call at end of each Swoole request to prevent leaks.

`static getInstance / setInstance`
Disabled - throw LogicException.

5. Loading Subsystem (on-demand modules)

This is the 'Demand' in Gated Demand Architecture. After ResolveStage names the service for a route, LoadStage computes which modules that service transitively needs and registers ONLY those into a fresh ModuleContainer.

`Loading/DependencyGraphCalculator.php`

Built from service-manifest.php. DFS over requires[] to produce the ordered module list for a service.

`resolve(string $service, array $additional=[]): DependencyGraph`
Compute the dependency graph for a service. $additional seeds extra domains (a project route's per-route requires[]) into just this request's graph.

`Loading/DependencyGraph.php`

Immutable value object: the resolved, ordered list of modules plus their manifest entries.

`moduleNames(): array`
Ordered domains to register (dependencies first).

`entry(string $service): array`
The manifest entry (module provider class, requires, etc.) for a service/domain.

`Loading/OnDemandLoader.php`

Turns a DependencyGraph into a live request-scoped ModuleContainer. Seeds request-scoped kernel services (Identity, DomainEventCollector, TransactionManager), then runs each module's register() under its own scope, plus the essential modules.

`load(DependencyGraph $g, Request $r): ModuleContainer`
HTTP path - extracts Identity from the request, then delegates to loadWithIdentity().

`loadWithIdentity(DependencyGraph $g, ?Identity $id): ModuleContainer`
Worker/test path - build the container with a pre-resolved Identity (null => guest).

6. Security Subsystem (pre-bootstrap gate)

SecurityGateway runs in SecurityStage before any module loads. Layers run in order; the first denial stops everything. Layers NEVER throw - they return a SecurityVerdict.

`Security/SecurityGateway.php`

Runs the configured SecurityLayerContract list and returns the final verdict.

`inspect(Request $r): SecurityVerdict`
Run each layer; return the first deny, or an allow carrying the resolved Identity.

`Security/Contracts/SecurityLayerContract.php`

The layer interface. Implemented by FirewallLayer, RateLimiterLayer, CsrfTokenLayer and a project's Auth layer.

`check(Request $r): SecurityVerdict`
Inspect the request and return allow/deny. Must never throw.

Security/SecurityVerdict.php

Immutable result of a security check.

`static allow(Request $r): self`
Proceed; may carry an Identity attached to the request.

`static deny(int $status, string $reason): self`
Stop with an HTTP status and reason.

`isAllowed(): bool / isDenied(): bool`
Branch on the verdict.

`statusCode(): int / reason(): string / identity(): ?Identity`
Read the denial code/reason or the resolved Identity.

Security/Identity.php

Immutable authenticated principal attached to the request and bound into every ModuleContainer. Authorization keys on (userId, tenantId, role/permission).

`hasRole(string $role): bool / hasPermission(string $perm): bool`
Authorization checks used in service-layer guards.

`isGuest(): bool`
True when userId is empty.

`static asUser(...) / asAdmin(...) / guest(): self`
Factory/test helpers for building identities.

Security/Layers/CsrfTokenLayer.php

Stateless HMAC-signed token CSRF (WordPress-nonce style) - nothing stored, no cookie trusted as the token, so cookie injection cannot bypass it.

`check(Request $r): SecurityVerdict`
Verify the token on unsafe methods unless the path is exempt.

`issue(Request $r, string $action=''): string`
Mint a token bound to the current request context.

`static make(string $secret, string $binding='', int $lifetime=43200, string $action=''): string`
Mint a token out-of-band.

`static valid(string $secret, string $token, string $binding='', int $lifetime=43200): bool`
Verify a token out-of-band (uses hash_equals).

7. HTTP Pipeline & Stages

Pipelines/Http/HttpPipeline.php

Assembles the stage onion (lazily, on first request) and runs it. Owns the hook slots and the FilterRegistry. Manifest collaborators (calculator/loader/matcher) are built on first request so a non-HTTP process pays no route/service manifest I/O.

`hook(string $slot, string $stageClass, int $priority=50): void`
Register a global stage in 'after.security'|'after.load'|'after.execute'. Lower priority runs first. Called from Module::boot().

`filter(string $alias, string $stageClass): void`
Publish a route-filter alias that routes opt into via filters[].

`handle(Request $r): Response`
Build (once) and run the pipeline for a request.

`Pipelines/Http/Contracts/HttpStageContract.php`

Every stage implements handle(Request, callable $next): Response - the onion contract.

`Pipelines/Http/RouteMatcher.php`

Compiled from route-manifest.php. Matches method+path (with {param} capture).

`match(string $method, string $path): ?array`
Return the matched route entry with extracted params, or null (404).

`Pipelines/Http/FilterRegistry.php`

Alias -> stage class registry shared by all plugins for declarative route filters.

`register(string $alias, string $stageClass): void`
Bind an alias; two plugins binding the same alias to different stages throws.

`has(string $alias): bool`
Whether an alias is known.

`resolve(string $alias, CoreContainer $core): HttpStageContract`
Instantiate the stage (from the core if bound, else new).

The stages (each: handle(Request,$next): Response)

`CorrelationIdStage`
Generate/propagate X-Correlation-ID; always first inside ErrorStage.

`SecurityStage(SecurityGateway)`
Run the gateway; on deny return the HTTP error, on allow attach Identity and continue.

`ResolveStage(RouteMatcher)`
Look up the route; attach route_entry + route_params, or return 404.

`LoadStage(DependencyGraphCalculator, OnDemandLoader)`
Build the dependency graph (incl. per-route requires[]), create the ModuleContainer, attach it to the request.

`RouteFilterStage(FilterRegistry, CoreContainer)`
Run the matched route's declared filters as a nested onion (left-to-right); exposes active_filters + filter_args.

`ExecuteStage`
Resolve the controller in its route scope (makeInScope); RequestAware controllers get setRequest() and receive route params only, others get ($request, ...params). Stamps X-Correlation-ID on the response.

`ErrorStage(ErrorPipeline)`
Outermost wrapper - catches every Throwable, feeds the ErrorPipeline, returns a safe Response (or debug page).

8. HTTP Engine Value Objects

Built on symfony/http-foundation today, but consumers must depend only on the kernel's own method surface so the planned dependency-free rewrite stays non-breaking. Request is immutable; there is exactly one Response type.

`Http/Request.php  (final, immutable)`

Every mutator returns a new instance; __clone deep-clones the bags for coroutine safety.

Construction & identity

`static capture(): static`
Build from PHP superglobals (FPM).

`static build(...) / createFromBase(SymfonyRequest): static`
Build a request manually / adapt a base Symfony request.

`method(): string / path(): string / decodedPath(): string`
HTTP verb (upper) and URL path.

`identity(): ?Identity / container(): ?ModuleContainer`
The request-scoped principal and DI container (set by the pipeline).

`attribute(string $k, $d=null): mixed / attributesAll(): array`
Pipeline-attached attributes (route_entry, domain, correlation_id...).

Input access (body+query merged, JSON auto-decoded)

`input(k,d) / all() / only(keys) / except(keys)`
Read merged input.

`query(k,d) / queryAll() / post(k,d) / body() / json()`
Source-specific access.

`has(k) / filled(k) / missing(k)`
Presence checks.

`boolean(k) / integer(k) / float(k) / string(k)`
Typed coercions.

`header(k,d) / hasHeader(k) / cookie(k,d) / hasCookie(k)`
Headers and cookies (case-insensitive).

`file(k): ?UploadedFile / hasFile(k) / isMethod(m)`
Uploads and method checks.

Immutable mutators (return new Request)

`withHeader / withAttribute / withIdentity / withContainer / merge / replace`
Each returns a new request; the original is untouched (Swoole-safe).

`Http/Response.php (final, single type)`

Named constructors only - controllers are typed : Response and must never return Symfony types.

`json / text / html / success / created / accepted / empty / noContent`
Success builders.

`notFound / unauthorized / forbidden / badRequest / conflict / tooManyRequests / unprocessable / serverError`
Error envelopes { error: { code, message[, fields] } }.

`redirect / permanentRedirect / back / jsonp`
Redirects and JSONP.

`stream / streamDownload / download / file`
Streaming + file responses - work on both FPM and Swoole.

`send(bool $flush=true): static`
Emit headers + body to the client.

`withHeader / withHeaders / withStatus / withCookie / withoutCookie`
Immutable mutators (via ManagesResponse trait).

`body() / status() / headers() / cookies() / isFile() / isStreamed() / filePath() / streamTo()`
Read accessors.

`Http/ other value objects`

Uri (PSR-7 immutable URI, ->Request::uri()), SiteUri (host-aware absolute URLs, ->site()), Negotiate (Accept- content negotiation), UploadedFile (FPM + Swoole safe), UserAgent (parsed UA), Method (enum: isSafe/isIdempotent/isCacheable/all), RequestAware (setRequest contract), Concerns/ManagesResponse (shared immutable response mutators).*

`Uri: getScheme/Host/Port/Path/Query + withScheme/withHost/withPath/withQuery...`
Immutable PSR-7 URI parts and copies.

`SiteUri::to(path, query) / asset(path) / base() / uri(path)`
Build absolute URLs without hardcoding the host.

`Negotiate::media/charset/encoding/language(supported, default)`
Pick the best supported representation from Accept-* headers.

`UploadedFile: clientName/clientMimeType/size/extension/isValid/contents + fromSwoole/createFromBase`
Safe upload handling across runtimes.

9. Events Subsystem

Two event types. Domain events are collected inside a transaction and discarded on rollback; integration events are dispatched ONLY after a successful commit.

`Events/DomainEventCollector.php`

Per-request buffer for in-transaction domain events.

`beginCollection(): void`
Start a collection window (in a service before the transaction).

`collect(DomainEventContract $e): void`
Buffer an event released by an entity.

`release(): array`
Return and clear the buffer (after commit).

`discard(): void`
Drop everything - called in every catch block so rollback leaves no phantom events.

`count(): int`
Number of buffered events.

`Events/EventBus.php`

Dispatches integration events to subscribed listeners, resolving them from the CoreContainer; listener failures are isolated.

`subscribe(string $eventName, string $listenerClass): void`
Register a listener (called from Module::boot()).

`dispatch(IntegrationEventContract $e): void`
Deliver to every subscriber for the event name.

`Events/Contracts/*`

DomainEventContract (marker), IntegrationEventContract (name/version/payload - primitives only), EventListenerContract (handle(IntegrationEventContract): void).

10. Database / Transactions

`Database/TransactionManager.php`

Thin wrapper over DatabasePort transaction control, bound per-request into the ModuleContainer. Services use the mandatory begin/commit/rollback shape around domain-event collection.

`begin(): void`
Open a transaction (DatabasePort::beginTransaction).

`commit(): void`
Commit. After this, services dispatch integration events.

`rollback(): void`
Roll back. Pair with collector->discard().

`inTransaction(): bool`
Whether a transaction is currently active.

11. Ports (infrastructure boundaries)

The kernel defines port interfaces; the project supplies adapters via withPorts(). A Repository may touch only DatabasePort; a Gateway only a vendor SDK. Ports are the seam that keeps the kernel infrastructure-independent.

- DatabasePort - query/queryOne/execute/lastInsertId + beginTransaction/commit/rollback/inTransaction.
- CachePort - get/set/delete/has/remember/increment/deletePattern/flush.
- QueuePort - push/later/size (enqueue jobs the WorkerLoop drains).
- MailPort - send/queue (view-based mail).
- SmsPort - send(to, message).
- StoragePort - store/storeStream/get/readStream/temporaryUrl/exists/delete (local or S3).
- HttpClientPort (+HttpClientResponse) - request/get/post/put/patch/delete for Gateways; response has status/json/ok/failed/throw.
- SessionPort - start/get/put/pull/push/flash/token/regenerate/invalidate/save (request-scoped).
- EncryptionPort - encrypt/decrypt/encryptString/decryptString.
- HashingPort - make/check/needsRehash (password hashing).

12. Worker Subsystem (background jobs)

`Pipelines/Worker/WorkerLoop.php`

The long-running job runner. Pulls payloads, verifies signatures, builds a per-job ModuleContainer (so jobs get TransactionManager, DomainEventCollector and their module DI), runs the job, and applies retry/dead-letter semantics.

`run(callable $puller, int $maxIterations=0): void`
Loop: pull a JobPayload (or idle-backoff on null) and process it. 0 = run forever until stop().

`stop(): void`
Signal the loop to exit after the current job.

`Pipelines/Worker/WorkerPipeline.php`

Job-name registry (compiled from job manifests / Module::boot()).

`job(string $name, string $jobClass): void`
Register a named job.

`resolve(string $name): ?string`
Map a job name to its class.

`jobs(): array`
All registered jobs.

`Pipelines/Worker/JobPayload.php / JobResult.php / JobContract.php`

Payload = immutable job envelope; Result = success/skipped outcome; Contract = the job interface.

`JobPayload: jobId/jobClass/data/queue/attempts/maxAttempts/enqueuedAt/signature`
Read the envelope fields.

`JobPayload::isSignatureValid(secret) / hasExceededMaxAttempts()`
Tamper check and retry-budget check.

`JobResult::success(data) / skipped(reason)`
Build outcomes; isSuccess/isSkipped/skipReason/data read them.

`JobContract::handle(JobPayload): JobResult`
Run the job (throw to retry).

`JobContract::failed(JobPayload, Throwable): void`
Called after max retries (dead-letter hook).

`Pipelines/Worker/Retry/*`

*RetryStrategyContract::delayFor(int $attempt): int with three implementations: Exponential (base*2^(n-1), capped, optional jitter), Linear (base*n, capped), Fixed (constant).*

13. CLI Subsystem

`Pipelines/Cli/CliPipeline.php`

Wraps the php-io-cli CLIApplication. Module commands extend AbstractCommand and are registered by class-string; CliPipeline instantiates them via the CoreContainer for port injection.

`command(string|AbstractCommand $c): void`
Register a command (class-string or instance).

`defer(\Closure $register): void`
Defer command registration until run() (lazy wiring).

`commands(): array`
All registered commands.

`application(): CLIApplication`
The underlying CLI app.

`container(): CoreContainer`
The core container backing command resolution.

`run(array $argv): int`
Dispatch the command and return the exit code.

`Pipelines/Cli/ (deprecated shims)`

Arguments, Output and Contracts/CommandContract are @deprecated - extend AbstractCommand from php-io-cli instead. They remain only for backward compatibility.

14. Error Subsystem

The inner of the two error nets (the outer is the project's ErrorGuard). ErrorStage feeds every Throwable into the ErrorPipeline which classifies severity and fans out to notifiers; FileNotifier is the guaranteed fallback.

`Error/ErrorPipeline.php`

Routes a classified error to notifiers per severity rules.

`static notifiers(array $n): self / static default(): self`
Build a pipeline from notifiers, or the default set.

`fallback(NotifierContract $n): self`
Set the always-runs fallback (FileNotifier).

`rules(array $rules): self`
Map severity -> notifier names (e.g. critical => [slack,mail,database,file]).

`consume(ErrorContext $ctx): void`
Classify and dispatch to the matching notifiers.

`Error/ErrorContext.php / ErrorClassifier.php`

ErrorContext is the immutable error snapshot; ErrorClassifier maps a Throwable to a severity.

`ErrorContext::fromThrowable(...) / toArray()`
Build the snapshot from a Throwable / serialize it for logs.

`ErrorClassifier::severityFor(Throwable): string`
Return 'critical' | 'warning' | 'info' (constants on the class).

`Error/Notifiers/*  (NotifierContract: name(), notify(ErrorContext))`

SlackNotifier, MailNotifier, DatabaseErrorLogger and FileNotifier (always runs - guaranteed fallback). DebugPageRenderer::renderHtml/renderCli produce the dependency-free debug page (debug builds only).

15. Exceptions & Support

FrameworkException is the abstract base (message, layer, context, previous). Each layer throws its own subclass, mapping to an HTTP status and severity.

- SecurityException 401/403 (warning) - but security LAYERS return deny(), they do not throw.
- DomainException 422 (info) | ServiceException 422/500 (warning).
- RepositoryException 500 (critical) - translate \PDOException here; OptimisticLockException extends it.
- GatewayException 502 (critical) - translate ALL vendor exceptions here.
- KernelException 500 (critical) | ValidationException 422 with field errors.
- BootFailureException / CircularDependencyException / ScopeViolationException - thrown by boot & DI scope enforcement.

`Support/Paths.php  (static path resolver)`

Resolves runtime roots, honouring the project path set in build().

`setBase(string) / setProject(?string)`
Set the resolution roots (called by Kernel::build).

`base / project / storage / var / logs / cache / userdata / config (string $append=''): string`
Return absolute paths under the corresponding root, falling back to base roots when no project is set.

`Support/helpers.php  (global functions)`

Procedural helpers (e.g. env(), config()). env() is the canonical .env reader - reads $_ENV/$_SERVER then getenv(); never use getenv() directly for .env values in first-party code.

`RuntimeMode.php  (enum)`

Http | Cli | Worker - the surface a kernel materialized for, returned by Kernel::mode().

16. Putting It Together - a Worked Request

A POST /api/invoices with a bearer token, end to end:

- 1. index.php resolves the DomainContext from the Host, requires the project bootstrap, gets the built kernel, and calls $kernel->http()->handle(Request::capture()).
- 2. http() materializes once (pipelines built, modules booted, core frozen).
- 3. ErrorStage wraps everything. CorrelationIdStage stamps the trace id.
- 4. SecurityStage runs the gateway: Firewall -> RateLimiter -> CSRF -> Auth layer. The Auth layer validates the JWT and returns allow(Identity); a failure would deny(401) here with zero module cost.
- 5. ResolveStage matches POST /api/invoices in route-manifest.php, attaching route_entry (handler InvoiceController@create, solves invoice.generation) and route_params.
- 6. LoadStage asks DependencyGraphCalculator for invoice.generation's graph, OnDemandLoader registers ONLY those modules into a fresh ModuleContainer (with Identity, TransactionManager, DomainEventCollector), and attaches it to the request.
- 7. RouteFilterStage runs the route's filters[] (e.g. auth, throttle).
- 8. ExecuteStage resolves InvoiceController in scope invoice.generation, calls create(); the controller builds a DTO and calls InvoiceServiceContract::create(); the service runs begin/commit, collects domain events, and

dispatches the integration event AFTER commit via EventBus.

- 9. The Response bubbles back out; ExecuteStage stamps X-Correlation-ID; send() emits it.
- 10. Any Throwable anywhere is caught by ErrorStage -> ErrorPipeline -> notifiers, returning a safe envelope.

That single path exercises nearly every subsystem in this document - which is exactly the point of GDA: a predictable, secure, minimal-cost flow where the kernel orchestrates and the modules supply the behaviour.